

Chapitre VII:

Les Triggers



- Les déclencheurs (ou triggers) caractérisent les bases de données actives.
- Une base de données active est capable de réagir à des événements et se fonde sur les règles **E-C-A** (**Evènement, Condition, Action**) dont le principe est d'exécuter une **action** suite à un **évènement** lorsqu'une **condition** est satisfaite.

Les déclencheurs (triggers) implantent ces règles.

- On distingue deux types de déclencheurs:
 - Les **déclencheurs applicatifs** qui sont créés et manipulés au niveau d'une application et de ce fait sont invisibles en dehors de celle-ci. Ces déclencheurs ne sont déclenchés que si la BD est manipulée à travers une application.
 - Les **déclencheurs de bases de données** sont stockés dans le dictionnaire de données du SGBD et associés à des événements sur des bases. Ces déclencheurs se déclenche automatiquement à chaque fois son événement associé est exécuté.



- ▶ Oracle a implanté le concept de trigger pour permettre :
 - Une gestion événementielle transparente.
 - Une mise à jour automatique cohérente.
 - Une sécurité renforcée des données de la base.
 - Une maintenabilité facile des tables.

- ▶ Par exemple, la mise à jour d'une table peut s'interpréter comme un événement déclencheur d'une ou plusieurs actions sur d'autres tables. A cet événement est associé un traitement appelé Trigger et qui est défini pour assurer la cohérence des données



La **suppression d'une ligne commande** doit engendrer la mise à jour du montant commande. Dans ce cas, on définit **un trigger de suppression sur la table ligne_commande**.

Ce trigger va mettre à jour le montant de la commande associée à la ligne supprimée en le diminuant du montant ligne.

Cet exemple montre en quoi les triggers permettent d'assurer la cohérence de données



- ▶ Les triggers ressemblent aux sous-programmes stockés essentiellement sur les points suivants:
 - Sont des blocs PL/SQL nommés.
 - Objets stockés dans la méta-base Oracle sous forme de code source, et p-code à partir de PL/SQL 2.3.
 - Peuvent être appelés des sous-programmes stockés.
- ▶ Il est différent sur les points suivants:
 - Exécutés implicitement au moment où l'évènement de déclenchement se produit.
 - Ne sont pas paramétrable.
 - Peuvent être désactivés.



- Un trigger est une procédure stockée dans la base de données. Il est associé à un événement de mise à jour sur une table (insertion, modification ou suppression), exécuté implicitement par le noyau chaque fois que l'événement qu'il représente se produit.
- Les commandes SQL déclenchant un trigger sont : **Insert**, **Delete**, et **Update** ou une combinaison de ces commandes.

```
Create [or replace] trigger <nom_trigger>
{Before | After}
{delete | insert | update of colonne }
[ Or {delete | insert | update of colonne }..]
On <nom_table>
[ For each row ]
[ when condition ]
Block PL/SQL
```

Les actions menées par le trigger sur la base de données sont décrites par le block PL/SQL.



```
Create [or replace] trigger <nom_trigger>  
{Before | After}  
{delete | insert | update of colonne }  
[ Or {delete | insert | update of colonne }..]  
On <nom_table>  
[ For each row ]  
[ when condition ]  
Block PL/SQL
```

- **After** spécifie que les actions du trigger sont exécutées après la commande de l'événement.
- **Before** spécifie que les actions sont exécutées avant la commande événement.
- **Nom_table** : c'est la table sur laquelle le trigger est défini. Si l'évènement est exécuté sur cette table, le trigger sera déclenché.
- Évènement : **INSERT, UPDATE, DELETE**: c'est l'évènement déclenchant l'exécution du trigger.

Il peut être une commande LMD ou une combinaison utilisant **OR**.



```
Create [or replace] trigger <nom_trigger>
{Before | After}
{delete | insert | update of colonne }
[ Or {delete | insert | update of colonne }..]
On <nom_table>
[ For each row ]
[ when condition ]
Block PL/SQL
```

- **For each row** utilisé pour les triggers de niveau ligne (**Row Level**), indique que les actions du trigger sont exécutées pour chaque ligne mise à jour par la commande associée (**Insert|Update|Delete**).

Autrement le trigger est de niveau instruction c-à-d il s'exécute une seule fois quelque soit le nombre de n-uplets affectés par l'évènement déclenchant (c'est le niveau par défaut).



```
Create [or replace] trigger <nom_trigger>
{Before | After}
{delete | insert | update of colonne }
[ Or {delete | insert | update of colonne }..]
On <nom_table>
[ For each row ]
[ when condition ]
Block PL/SQL
```

- **When condition** c'est une **option de la clause FOR EACH ROW** (valable uniquement pour les triggers de niveau ligne), introduit une condition selon laquelle le corps du trigger peut être exécuté ou simplement abandonné.



Les pseudo-records OLD et NEW & les prédicats INSERTING, UPDATING et DELETING

- ▶ Le trigger se déclenche chaque fois qu'on ajoute un nouveau tuple (**if inserting**) ou qu'on supprime (**if deleting**) un tuple d'une table de la base ou qu'on modifie un tuple (**if updating**).
- ▶ Les champs du tuple ajouté sont accessibles via le quantificateur système : **new.nom_colonne**.
- ▶ Les champs du tuple supprimé sont accessibles via le quantificateur : **old.nom_colonne**.
- ▶ Les champs du tuple modifié sont accessibles via le quantificateur : **old.nom_colonne** ou via le quantificateur : **new.nom_colonne**.



► Une fois créé, un trigger doit être **mis en service** pour qu'il puisse être déclenché systématiquement chaque fois que son événement se manifeste. Son activation se fait par la commande :

```
Alter trigger nom_trigger enable ;
```

► Pour le **désactiver** (il est présent dans la base mais hors service), on exécute la commande :

```
Alter trigger nom_trigger disable ;
```



► Pour que le trigger prenne en considération les modifications apportées sur la base (modification ou suppression de certaines tables) il faut le compiler par la commande :

```
Alter trigger nom_trigger compile ;
```

► Pour supprimer un trigger de la base, on utilise la commande :

```
Drop trigger nom_trigger;
```



- Les commandes SQL utilisées dans le corps d'un déclencheur ne peuvent:
 - Lire ou modifier aucune des tables en mutation de l'évènement déclenchant.
 - Modifier la valeur d'une colonne définie avec l'une des contraintes **PRIMARY KEY**, **UNIQUE** ou **FOREIGN KEY**.
 - Cependant, elles peuvent modifier les autres colonnes.



Soit la BD suivante :

Article (nart, desart, qteStk, pu)
Client (ncl, nomcl, adrcl)
Commande (nc, datec, #ncl, mntc)
Ligne_Commande (#nc, #nart, qtec)

Créer un trigger qui permet de mettre à jour automatiquement le montant de la commande pour chaque ligne de commande ajoutée ou supprimée ou modifiée (qtec).